13. SDK 사용 예제 및 베스트 프랙티스

개요

SP1 SDK는 SP1 zkVM과 상호작용하기 위한 고수준 인터페이스를 제공합니다. 이 문서는 기본적인 프로그램 작성부터 복잡한 증명 생성까지의 전체 과정을 예제와 함께 설명하고, 실제 개발에서 유용한 베스트 프랙티스를 제시합니다.

13.1 기본 SDK 사용법

프로젝트 구조 설정

```
my-sp1-project/
├── program/ # SP1 프로그램 (zkVM에서 실행)
| ├── Cargo.toml
| └── src/
| └── main.rs
└── script/ # 증명 생성 스크립트 (호스트에서 실행)
├── Cargo.toml
├── build.rs
└── src/
└── main.rs
```

기본 프로그램 작성

program/Cargo.toml

```toml
[package]
name = "my-program"
version = "1.0.0"
edition = "2021"

[dependencies]
sp1-zkvm = { version = "4.0.0", features = ["embedded"] }
```

program/src/main.rs

```rust
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
// 입력 데이터 읽기
let n = sp1_zkvm::io::read::<u32>();
// 계산 수행
let result = fibonacci(n);
// 결과를 공개 값으로 커밋
sp1_zkvm::io::commit(&result);
}
```

```
fn fibonacci(n: u32) -> u32 {
if n <= 1 {
return n;
}
let mut a = 0;
let mut b = 1;
for _ in 2..=n {
let temp = a + b;
a = b;
b = temp;
}
b
}
```

증명 생성 스크립트 작성

script/Cargo.toml

```
[package]
name = "my-script"
version = "1.0.0"
edition = "2021"

[dependencies]
```

```
sp1-sdk = "4.0.0"


[build-dependencies]

sp1-build = "4.0.0"


[features]

cuda = ["sp1-sdk/cuda"]
```

script/build.rs

```
fn  main() {

sp1_build::build_program("../program");

}
```

script/src/main.rs

```
use sp1_sdk::{include_elf, ProverClient, SP1Stdin};


// 컴파일된 프로그램 ELF 포함

const ELF: &[u8] = include_elf!("my-program");


fn  main() {

// 로깅 설정

sp1_sdk::utils::setup_logger();
```

```rust
// 입력 데이터 준비
let mut stdin = SP1Stdin::new();
stdin.write(&10u32);
// 프로버 클라이언트 생성
let client = ProverClient::from_env();
// 프로그램 설정 (키 생성)
let (pk, vk) = client.setup(ELF);
// 증명 생성
let proof = client.prove(&pk, &stdin).run().unwrap();
// 결과 읽기
let result = proof.public_values.read::<u32>();
println!("Fibonacci(10) = {}", result);
// 증명 검증
client.verify(&proof, &vk).expect("verification failed");
println!("Proof generated and verified successfully!");
}
```

13.2 다양한 프로버 타입 사용

CPU 프로버 (기본)

```rust
use sp1_sdk::{ProverClient, Prover};
```

```rust
// 환경 변수에서 자동 선택
let client = ProverClient::from_env();

// 명시적으로 CPU 프로버 사용
let client = ProverClient::builder().cpu().build();
```

CUDA 프로버 (GPU 가속)

```rust
use sp1_sdk::{ProverClient, Prover};

// CUDA 프로버 사용
let client = ProverClient::builder().cuda().build();

// 환경 변수로 설정
std::env::set_var("SP1_PROVER", "cuda");
let client = ProverClient::from_env();
```

네트워크 프로버

```rust
use sp1_sdk::{ProverClient, Prover};

#[cfg(feature = "network")]
{
```

```
// 네트워크 프로버 사용
let client = ProverClient::builder()
.network()
.private_key("your-private-key")
.rpc_url("https://rpc.succinct.xyz")
.build();
}
```

모크 프로버 (테스트용)

```
use sp1_sdk::{ProverClient, Prover};

// 빠른 테스트를 위한 모크 프로버
let client = ProverClient::builder().mock().build();
```

13.3 다양한 증명 타입

Core 증명 (기본 STARK)

```
let proof = client.prove(&pk, &stdin).run().unwrap();
```

압축 증명

```
let proof = client.prove(&pk, &stdin).compressed().run().unwrap();
```

PLONK 증명

```
let proof = client.prove(&pk, &stdin).plonk().run().unwrap();
```

Groth16 증명

```
let proof = client.prove(&pk, &stdin).groth16().run().unwrap();
```

13.4 복잡한 데이터 타입 처리

구조체 직렬화

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Debug, Clone)]
struct Person {
name: String,
age: u32,
email: String,
}
```

```rust
// 프로그램에서
let person = sp1_zkvm::io::read::<Person>();
sp1_zkvm::io::commit(&person);


// 스크립트에서
let person = Person {
name: "Alice".to_string(),
age: 30,
email: "alice@example.com".to_string(),
};


let mut stdin = SP1Stdin::new();
stdin.write(&person);


let proof = client.prove(&pk, &stdin).run().unwrap();
let result = proof.public_values.read::<Person>();
```

벡터 데이터 처리

```rust
// 프로그램에서
let numbers = sp1_zkvm::io::read::<Vec<u32>>();
let sum: u32 = numbers.iter().sum();
sp1_zkvm::io::commit(&sum);
```

```rust
// 스크립트에서
let numbers = vec![1, 2, 3, 4, 5];
let mut stdin = SP1Stdin::new();
stdin.write(&numbers);
```

JSON 데이터 처리

```rust
use serde_json::Value;

// 프로그램에서
let json_str = sp1_zkvm::io::read::<String>();
let json: Value = serde_json::from_str(&json_str).unwrap();
let name = json["name"].as_str().unwrap();
sp1_zkvm::io::commit(&name.to_string());

// 스크립트에서
let json_data = r#"{"name": "Bob", "age": 25}"#;
let mut stdin = SP1Stdin::new();
stdin.write(&json_data.to_string());
```

13.5 고급 사용 사례

암호화 알고리즘 검증

```
// RSA 서명 검증 예제
use rsa::{RsaPublicKey, pkcs1v15::VerifyingKey, signature::Verifier};
use sha2::Sha256;

// 프로그램에서
let public_key_der = sp1_zkvm::io::read::<Vec<u8>>();
let message = sp1_zkvm::io::read::<Vec<u8>>();
let signature = sp1_zkvm::io::read::<Vec<u8>>();

let public_key = RsaPublicKey::from_public_key_der(&public_key_der).unwrap();
let verifying_key = VerifyingKey::<Sha256>::new(public_key);

let is_valid = verifying_key.verify(&message, &signature.try_into().unwrap()).i
sp1_zkvm::io::commit(&is_valid);
```

머클 트리 검증

```
use sha2::{Sha256, Digest};

#[derive(Serialize, Deserialize)]
struct MerkleProof {
```

```
leaf: [u8; 32],
proof: Vec<[u8; 32]>,
indices: Vec<bool>, // true = right, false = left
root: [u8; 32],
}

// 프로그램에서
let proof = sp1_zkvm::io::read::<MerkleProof>();
let is_valid = verify_merkle_proof(&proof);
sp1_zkvm::io::commit(&is_valid);

fn verify_merkle_proof(proof: &MerkleProof) -> bool {
let mut current_hash = proof.leaf;
for (sibling, is_right) in proof.proof.iter().zip(proof.indices.iter()) {
let mut hasher = Sha256::new();
if *is_right {
hasher.update(current_hash);
hasher.update(sibling);
} else {
hasher.update(sibling);
hasher.update(current_hash);
}
current_hash = hasher.finalize().into();
```

```
}
current_hash == proof.root
}
```

증명 집계 (Aggregation)

```
use sp1_sdk::{SP1ProofWithPublicValues, SP1VerifyingKey};

struct AggregationInput {
proof: SP1ProofWithPublicValues,
vk: SP1VerifyingKey,
}

// 여러 증명 생성
let proof1 = client.prove(&pk1, &stdin1).compressed().run().unwrap();
let proof2 = client.prove(&pk2, &stdin2).compressed().run().unwrap();
let proof3 = client.prove(&pk3, &stdin3).compressed().run().unwrap();

// 집계 프로그램 입력 준비
let inputs = vec![
AggregationInput { proof: proof1, vk: vk1 },
AggregationInput { proof: proof2, vk: vk2 },
AggregationInput { proof: proof3, vk: vk3 },
```

```
];

let mut stdin = SP1Stdin::new();

// 검증 키 해시 작성
let vkeys: Vec<[u32; 8]> = inputs.iter()
.map(|input| input.vk.hash_u32())
.collect();
stdin.write(&vkeys);

// 공개 값 작성
let public_values: Vec<Vec<u8>> = inputs.iter()
.map(|input| input.proof.public_values.to_vec())
.collect();
stdin.write(&public_values);

// 증명 데이터 작성
for input in inputs {
if let SP1Proof::Compressed(proof) = input.proof.proof {
stdin.write_proof(*proof, input.vk.vk);
}
}
```

```
// 집계된 증명 생성
let aggregated_proof = client.prove(&aggregation_pk, &stdin).plonk().run().unwr
```

13.6 성능 최적화 기법

실행 컨텍스트 설정

```
use sp1_sdk::{SP1Context, SP1ContextBuilder};

// 사이클 제한 설정
let context = SP1ContextBuilder::default()
.max_cycles(1_000_000)
.shard_size(1 << 22) // 4MB
.build();

let (output, report) = client.execute(ELF, &stdin)
.context(context)
.run()
.unwrap();
```

메모리 최적화

```
// 환경 변수로 메모리 사용량 조정
```

```
std::env::set_var("SP1_SHARD_SIZE", "1048576"); // 1MB
std::env::set_var("SP1_MAX_CYCLES", "10000000");

// 스트리밍 입력 사용
let mut stdin = SP1Stdin::new();
for chunk in large_data.chunks(1024) {
stdin.write(chunk);
}
```

병렬 처리 활용

```
use rayon::prelude::*;

// 여러 증명을 병렬로 생성
let proofs: Vec<_> = inputs.par_iter()
.map(|input| {
let client = ProverClient::from_env();
let (pk, _) = client.setup(ELF);
client.prove(&pk, input).run().unwrap()
})
.collect();
```

13.7 오류 처리 및 디버깅

실행 오류 처리

```
use sp1_sdk::{ExecutionReport, SP1VerificationError};

// 실행 전 검증
match client.execute(ELF, &stdin).run() {
Ok((output, report)) => {
println!("Execution successful: {} cycles", report.total_instruction_count());
}
Err(e) => {
eprintln!("Execution failed: {}", e);
return;
}
}

// 증명 생성 오류 처리
match client.prove(&pk, &stdin).run() {
Ok(proof) => {
// 증명 검증
match client.verify(&proof, &vk) {
Ok(_) => println!("Verification successful"),
Err(SP1VerificationError::InvalidProof) => {
eprintln!("Invalid proof");
```

```
}
Err(e) => eprintln!("Verification error: {}", e),
}
}
Err(e) => {
eprintln!("Proving failed: {}", e);
}
}
```

디버깅 도구

```
// 상세 로깅 활성화
std::env::set_var("RUST_LOG", "debug");
sp1_sdk::utils::setup_logger();

// 실행 통계 출력
let (_, report) = client.execute(ELF, &stdin).run().unwrap();
println!("Execution report: {}", report);

// 메모리 사용량 추적
std::env::set_var("SP1_MEMORY_TRACE", "1");

// 사이클 추적
```

```
std::env::set_var("SP1_CYCLE_TRACE", "1");
```

프로파일링

```
use std::time::Instant;

// 실행 시간 측정
let  start = Instant::now();
let (_, report) = client.execute(ELF, &stdin).run().unwrap();
let  execution_time = start.elapsed();

println!("Execution time: {:?}", execution_time);
println!("Cycles per second: {:.2}",
report.total_instruction_count() as f64 / execution_time.as_secs_f64());

// 증명 생성 시간 측정
let  start = Instant::now();
let  proof = client.prove(&pk, &stdin).run().unwrap();
let  proving_time = start.elapsed();

println!("Proving time: {:?}", proving_time);
```

13.8 테스트 전략

단위 테스트

```rust
#[cfg(test)]
mod tests {
use super::*;
use sp1_sdk::{ProverClient, SP1Stdin};


#[test]
fn test_fibonacci_program() {
let client = ProverClient::builder().mock().build();
let (pk, vk) = client.setup(ELF);
let mut stdin = SP1Stdin::new();
stdin.write(&10u32);
let proof = client.prove(&pk, &stdin).run().unwrap();
let result = proof.public_values.read::<u32>();
assert_eq!(result, 55); // fibonacci(10) = 55
client.verify(&proof, &vk).unwrap();
}
#[test]
fn test_edge_cases() {
let client = ProverClient::builder().mock().build();
let (pk, vk) = client.setup(ELF);
// 경계값 테스트
```

```
for n in [0, 1, 2, 100] {
let mut stdin = SP1Stdin::new();
stdin.write(&n);
let proof = client.prove(&pk, &stdin).run().unwrap();
client.verify(&proof, &vk).unwrap();
}
}
}
```

통합 테스트

```
#[test]
fn test_end_to_end_workflow() {
// 실제 프로버로 전체 워크플로우 테스트
let client = ProverClient::builder().cpu().build();
// 1. 프로그램 설정
let (pk, vk) = client.setup(ELF);
// 2. 다양한 입력으로 테스트
let test_cases = vec![5, 10, 15, 20];
for n in test_cases {
let mut stdin = SP1Stdin::new();
stdin.write(&n);
// 3. 증명 생성
```

```
let proof = client.prove(&pk, &stdin).run().unwrap();

// 4. 검증

client.verify(&proof, &vk).unwrap();

// 5. 직렬화/역직렬화 테스트

proof.save(&format!("proof_{}.bin", n)).unwrap();

let loaded_proof = SP1ProofWithPublicValues::load(&format!("proof_{}.bin", n)).

client.verify(&loaded_proof, &vk).unwrap();

}

}
```

13.9 베스트 프랙티스

1. 프로젝트 구조

- **모듈화**: 프로그램과 스크립트를 분리
- **재사용성**: 공통 타입을 별도 라이브러리로 분리
- **테스트**: 각 컴포넌트에 대한 충분한 테스트 작성

2. 성능 최적화

- **적절한 프로버 선택**: 개발 시 mock, 테스트 시 CPU, 프로덕션 시 CUDA
- **메모리 관리**: 큰 데이터는 청크 단위로 처리
- **병렬 처리**: 독립적인 증명은 병렬로 생성

3. 보안 고려사항

- **입력 검증**: 프로그램 내에서 모든 입력 검증

- **키 관리**: 검증 키의 안전한 저장 및 배포
- **증명 무결성**: 증명 데이터의 변조 방지

4. 개발 워크플로우

- **점진적 개발**: 작은 기능부터 시작하여 점진적 확장
- **지속적 테스트**: 변경 사항마다 자동화된 테스트 실행
- **성능 모니터링**: 정기적인 성능 측정 및 최적화

5. 문제 해결

- **로깅 활용**: 적절한 로그 레벨 설정으로 디버깅 정보 수집
- **단계별 검증**: 실행 → 증명 생성 → 검증 순서로 단계별 확인
- **환경 분리**: 개발, 테스트, 프로덕션 환경 구분

이 문서는 SP1 SDK의 다양한 사용 사례와 실제 개발에서 유용한 패턴들을 제시하여, 개발자가 효율적이고 안정적인 SP1 애플리케이션을 구축할 수 있도록 돕습니다.